AP COMPUTER SCIENCE A · UNIT 2 WORKSHEET

Selection and Iteration Algorithms

CED Topic 2.9, with nested loops (2.11) · AP Java Subset only · Mr. Del Valle · Doral Academy Preparatory School

Name: _____ Period: _____ Date: _____

Instructions

- Work on paper. Do not use an IDE or a compiler.
- Part A: build a trace table for every loop, even when the worksheet does not provide one.
- Part B: each method compiles but does not do what its description says. Find the error.
- Parts C and D: use only the AP Java Subset. Do not use `break`, `continue`, `do-while`, or `switch`.

Part	Skill	Exercises
A	Trace It: predict the exact output	1–6
B	Find and Fix: locate and correct the error	7–10
C	Complete the Code: fill in the blanks	11–14
D	Write It: write the complete method	15–20

Part A · Trace It

Determine the exact output of each code segment.

1. What is printed?

CODE

```
int x = 0;
int y = 0;
for (int k = 1; k <= 12; k++)
{
    if (k % 2 == 0 && k % 3 == 0)
    {
        x += k;
    }
    else if (k % 3 == 0)
    {
        y++;
    }
}
System.out.println(x + " " + y);
```

Output: _____

2. What is printed? Write each line of output.

CODE

```
for (int r = 1; r <= 4; r++)
{
    for (int c = 1; c <= 4; c++)
    {
        if (c < r)
        {
            System.out.print("-");
        }
        else if (c == r)
        {
            System.out.print(r);
        }
        else
        {
            System.out.print("o");
        }
    }
    System.out.println();
}
```

Output:

3. What is printed?

CODE

```
int num = 908172;
int result = 0;
while (num > 0)
{
    int d = num % 10;
    if (d > 4)
    {
        if (d % 2 == 0)
        {
            result += 10;
        }
        else
        {
            result += 1;
        }
    }
    num /= 10;
}
System.out.println(result);
```

Output: _____

4. Answer both parts.

CODE

```
int count = 0;
for (int i = 1; i <= 6; i++)
{
    for (int j = i; j <= 6; j++)
    {
        if ((i + j) % 2 == 0)
        {
            count++;
        }
    }
}
System.out.println(count);
```

(a) How many times is the condition `(i + j) % 2 == 0` evaluated?

_____

(b) What is printed? _____

5. What is printed? Complete the trace table as you go.

CODE

```
int n = 50;
int count = 0;
while (n > 0)
{
    if (n % 3 == 0)
    {
        n -= 7;
    }
    else
    {
        n -= 4;
    }
    count++;
}
System.out.println(n + " " + count);
```

Pass	n at start	n % 3 == 0 ?	n after	count after

Output: _____

6. What is printed? Complete the trace table as you go.

CODE

```
int k = 10;
boolean found = false;
while (k < 40 && !found)
{
    if (k % 7 == 0 && k % 2 == 1)
    {
        found = true;
    }
    else
    {
        k += 3;
    }
}
System.out.println(k + " " + found);
```

Pass	k	k % 7 == 0 ?	k % 2 == 1 ?	found

Output: _____

Part B · Find and Fix

Each method compiles and runs, but it does not do what its description says. Answer all three parts for each exercise.

7. `averageNonMultiples`

The method should return the average of all integers from 1 to `n` that are **not** multiples of 3.

CODE

```
public static double averageNonMultiples(int n)
{
    int sum = 0;
    int count = 0;
    for (int k = 1; k <= n; k++)
    {
        if (k % 3 != 0)
        {
            sum += k;
            count++;
        }
    }
    return sum / count;
}
```

(a) What does `averageNonMultiples(7)` return as written? _____

(b) What should it return? _____

(c) Rewrite the incorrect line(s) so the method works:

8. `largestValue`

The method should return the largest value of `k * k - 10 * k` for every `k` from 1 to `n`. Precondition: `n >= 1`.

CODE

```
public static int largestValue(int n)
{
    int largest = 0;
    for (int k = 1; k <= n; k++)
    {
        int value = k * k - 10 * k;
        if (value > largest)
        {
            largest = value;
        }
    }
    return largest;
}
```

(a) What does `largestValue(5)` return as written? _____

(b) What should it return? _____

(c) Rewrite the incorrect line(s) so the method works:

9. `printPrimes`

The method should print every prime number from 2 to `limit`, separated by spaces.

CODE

```
public static void printPrimes(int limit)
{
    boolean prime = true;
    for (int n = 2; n <= limit; n++)
    {
        for (int d = 2; d < n; d++)
        {
            if (n % d == 0)
            {
                prime = false;
            }
        }
        if (prime)
        {
            System.out.print(n + " ");
        }
    }
    System.out.println();
}
```

(a) What does `printPrimes(15)` print as written? _____

(b) What should it print? _____

(c) Rewrite the incorrect line(s) so the method works:

10. `totalPoints`

Each integer from 1 to `n` earns points: multiples of **both** 3 and 5 earn 10 points, other multiples of 3 earn 3 points, other multiples of 5 earn 5 points, and every other number earns 0. The method should return the total.

CODE

```
public static int totalPoints(int n)
{
    int total = 0;
    for (int k = 1; k <= n; k++)
    {
        if (k % 3 == 0)
        {
            total += 3;
        }
        else if (k % 5 == 0)
        {
            total += 5;
        }
        else if (k % 3 == 0 && k % 5 == 0)
        {
            total += 10;
        }
    }
    return total;
}
```

(a) What does `totalPoints(30)` return as written? _____

(b) What should it return? _____

(c) Rewrite the incorrect line(s) so the method works:

Part C · Complete the Code

Fill in each blank so the method works for every input that meets its precondition.

11. `countDigit`

Returns how many times the digit `target` appears in `n`. Precondition: `n > 0` and `0 <= target <= 9`.

Call	Result
`countDigit(7071977, 7)`	`4`
`countDigit(12345, 9)`	`0`
`countDigit(100100, 0)`	`4`

CODE

```
public static int countDigit(int n, int target)
{
    int count = 0;
    while ( _______(a)_______ )
    {
        if ( _______(b)_______ )
        {
            count++;
        }
        _______(c)_______ ;
    }
    return count;
}
```

(a) _____

(b) _____

(c) _____

12. `smallestFactor`

Returns the smallest integer greater than 1 that divides `n` evenly. Precondition: `n >= 2`.

Call	Result
`smallestFactor(91)`	`7`
`smallestFactor(13)`	`13`
`smallestFactor(45)`	`3`

CODE

```
public static int smallestFactor(int n)
{
    int d = _______(a)_______ ;
    while ( _______(b)_______ )
    {
        _______(c)_______ ;
    }
    return d;
}
```

(a) _____

(b) _____

(c) _____

13. `printFrame`

Prints a rectangle with `rows` rows and `cols` columns. Characters on the outer edge are `#`; characters inside are `.`. Precondition: `rows >= 2` and `cols >= 2`.

Call	Result
`printFrame(4, 6)`	see output below

OUTPUT

```
######
#....#
#....#
######
```

CODE

```
public static void printFrame(int rows, int cols)
{
    for (int r = 1; r <= rows; r++)
    {
        for (int c = 1; c <= cols; c++)
        {
            if ( _____________________(a)_____________________ )
            {
                System.out.print("#");
            }
            else
            {
                System.out.print(".");
            }
        }
        _______(b)_______ ;
    }
}
```

(a) _____

(b) _____

14. `longestRun`

Returns the length of the longest run of identical consecutive digits in `n`. Precondition: `n > 0`.

Call	Result
`longestRun(1122233)`	`3`
`longestRun(98765)`	`1`
`longestRun(4444114)`	`4`

CODE

```
public static int longestRun(int n)
{
    int prev = -1;
    int run = 0;
    int best = 0;
    while (n > 0)
    {
        int d = n % 10;
        if ( _______(a)_______ )
        {
            run++;
        }
        else
        {
            _______(b)_______ ;
        }
        if ( _______(c)_______ )
        {
            best = run;
        }
        _______(d)_______ ;
        n /= 10;
    }
    return best;
}
```

(a) _____

(b) _____

(c) _____

(d) _____

Part D · Write It

Write the complete method. Each one needs at least one loop and at least one conditional. Trace your method with the examples before you move on.

15. `sumOfOddSquares`

Returns the sum of the squares of all odd integers from 1 to `n`. Precondition: `n >= 1`.

Call	Result
`sumOfOddSquares(5)`	`35` (1 + 9 + 25)
`sumOfOddSquares(6)`	`35`
`sumOfOddSquares(1)`	`1`

CODE

```
public static int sumOfOddSquares(int n)
```

16. `largestDigit`

Returns the largest digit in `n`. Precondition: `n > 0`.

Call	Result
`largestDigit(38271)`	`8`
`largestDigit(5)`	`5`
`largestDigit(1000)`	`1`

CODE

```
public static int largestDigit(int n)
```

17. `isNumberPalindrome`

Returns `true` if `n` reads the same forward and backward, and `false` otherwise. Precondition: `n > 0`.

Call	Result
`isNumberPalindrome(12321)`	`true`
`isNumberPalindrome(1231)`	`false`
`isNumberPalindrome(7)`	`true`
`isNumberPalindrome(120)`	`false`

CODE

```
public static boolean isNumberPalindrome(int n)
```

18. `nextPrime`

Returns the smallest prime number that is **greater than** `n`. Precondition: `n >= 0`. Use a nested loop. You may write and call a helper method.

Call	Result
`nextPrime(13)`	`17`
`nextPrime(1)`	`2`
`nextPrime(24)`	`29`
`nextPrime(89)`	`97`

CODE

```
public static int nextPrime(int n)
```

19. `printStaircase`

Prints `n` rows. Row `r` (from 1 to `n`) contains `n - r` dots followed by `r` `#` characters. Use nested loops with an `if` statement inside the inner loop.

Call	Result
`printStaircase(4)`	see output below

OUTPUT

```
...#
..##
.###
####
```

CODE

```
public static void printStaircase(int n)
```

20. `countAbundant`

A positive integer is **abundant** if the sum of its divisors smaller than itself is greater than the number. For example, 12 is abundant because 1 + 2 + 3 + 4 + 6 = 16, and 16 > 12. Returns how many integers from 1 to `limit` are abundant.

Call	Result
`countAbundant(30)`	`5` (12, 18, 20, 24, 30)
`countAbundant(11)`	`0`
`countAbundant(12)`	`1`

CODE

```
public static int countAbundant(int limit)
```